■ LOGIC MASTER

SwiftUI iOS Game — Complete Core Source Code

Project files included

1. LogicMasterApp.swift — app entry point
2. ContentView.swift — home screen and Start Game button
3. LevelView.swift — Easy, Medium, and Hard level selection
4. GameView.swift — questions, answer checking, scoring, progress, results, and replay

How to use this PDF

In Xcode, open your LogicMaster project and paste each code section into the file with the matching name. If Xcode already created LogicMasterApp.swift, keep only one @main app entry point. This document reconstructs the core game files from the project details available; it may not include every later UI or asset change made on your Mac.

Project flow: Home → Choose Level → Play Questions → Final Score → Play Again

1. LogicMasterApp.swift

```
import SwiftUI

@main
struct LogicMasterApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

2. ContentView.swift

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        NavigationStack {
            VStack(spacing: 24) {
                Spacer()

                Text("■")
                    .font(.system(size: 82))

                Text("Logic Master")
                    .font(.largeTitle)
                    .fontWeight(.bold)

                Text("Test your logic skills!")
                    .font(.headline)
                    .foregroundStyle(.secondary)

                NavigationLink {
                    LevelView()
                } label: {
                    Text("■ Start Game")
                        .font(.title2)
                        .fontWeight(.bold)
                        .frame(maxWidth: .infinity)
                        .padding()
                        .background(Color.blue)
                        .foregroundStyle(.white)
                        .clipShape(RoundedRectangle(cornerRadius: 15))
                }
                .padding(.horizontal, 28)

                Spacer()
            }
            .padding()
            .navigationTitle("Logic Master")
            .navigationBarTitleDisplayMode(.inline)
        }
    }
}

#Preview {
    ContentView()
}
```

3. LevelView.swift

```
import SwiftUI

struct LevelView: View {
    var body: some View {
        VStack(spacing: 22) {
            Text("Choose Level")
                .font(.largeTitle)
                .fontWeight(.bold)

            Text("Select a difficulty to begin.")
                .foregroundStyle(.secondary)

            levelLink(title: "■ Easy", level: "Easy", color: .green)
            levelLink(title: "■ Medium", level: "Medium", color: .orange)
            levelLink(title: "■ Hard", level: "Hard", color: .red)

            Spacer()
        }
        .padding()
        .navigationTitle("Levels")
    }

    private func levelLink(
        title: String,
        level: String,
        color: Color
    ) -> some View {
        NavigationLink {
            GameView(level: level)
        } label: {
            Text(title)
                .font(.title2)
                .fontWeight(.semibold)
                .frame(maxWidth: .infinity)
                .padding()
                .background(color)
                .foregroundStyle(.white)
                .clipShape(RoundedRectangle(cornerRadius: 14))
        }
    }
}

#Preview {
    NavigationStack {
        LevelView()
    }
}
```

4. GameView.swift

```
import SwiftUI

struct GameView: View {
    let level: String

    @State private var questionNumber = 0
    @State private var score = 0
    @State private var message = ""
    @State private var answerWasChecked = false
    @State private var gameOver = false

    private struct Question {
        let text: String
        let options: [String]
        let answer: String
        let explanation: String
    }

    private var questions: [Question] {
        switch level {
        case "Easy":
            return [
                Question(
                    text: "What comes next?\\n2, 4, 6, 8, ?",
                    options: ["9", "10", "11", "12"],
                    answer: "10",
                    explanation: "The numbers increase by 2."
                ),
                Question(
                    text: "What is 5 + 5?",
                    options: ["8", "9", "10", "11"],
                    answer: "10",
                    explanation: "5 + 5 = 10."
                ),
                Question(
                    text: "Which one is different?",
                    options: ["Apple", "Banana", "Carrot", "Mango"],
                    answer: "Carrot",
                    explanation: "Carrot is usually treated as a vegetable."
                )
            ]

        case "Medium":
            return [
                Question(
                    text: "What comes next?\\n3, 6, 12, 24, ?",
                    options: ["36", "42", "48", "50"],
                    answer: "48",
                    explanation: "Each number is multiplied by 2."
                ),
                Question(
                    text: "If all Bloops are Razzies and all Razzies are Lazzies, are
    all Bloops Lazzies?",
                    options: ["Yes", "No", "Cannot tell", "Only some"],
                    answer: "Yes",
                    explanation: "If each group belongs to the next group, Bloops are
    Lazzies."
                ),
                Question(
                    text: "A clock shows 3:00. What is the angle between its hands?",
                    options: ["45°", "60°", "90°", "180°"],
                    answer: "90°",
                    explanation: "At 3:00, the hands form a right angle."
                )
            ]

        default: // Hard
            return [
                Question(
                    text: "What comes next?\\n1, 1, 2, 3, 5, 8, ?",
                    options: ["11", "12", "13", "15"],
                    answer: "13",
                    explanation: "Each number is the sum of the two before it."
                ),
                Question(
                    text: "A farmer has 17 sheep. All but 9 run away. How many remain?",
```

```
                options: ["8", "9", "17", "0"],
                answer: "9",
                explanation: "'All but 9' means 9 sheep remain."
            ),
            Question(
                text: "What comes next?\\n2, 3, 5, 8, 12, ?",
                options: ["15", "16", "17", "18"],
                answer: "17",
                explanation: "The differences are +1, +2, +3, +4, then +5."
            ),
            Question(
                text: "Which number does not belong?",
                options: ["16", "25", "36", "48"],
                answer: "48",
                explanation: "16, 25, and 36 are perfect squares."
            ),
            Question(
                text: "You pass the person in second place in a race. What place are
 you in?",
                options: ["First", "Second", "Third", "Fourth"],
                answer: "Second",
                explanation: "You take the position of the person you passed."
            )
        ]
    }
}

private var currentQuestion: Question {
    questions[questionNumber]
}

var body: some View {
    VStack(spacing: 20) {
        if gameOver {
            completionView
        } else {
            HStack {
                Text("Level: \\(level)")
                Spacer()
                Text("Score: \\(score)")
            }
            .font(.headline)

            ProgressView(
                value: Double(questionNumber + 1),
                total: Double(questions.count)
            )

            Text("Question \\(questionNumber + 1) of \\(questions.count)")
                .font(.subheadline)
                .foregroundStyle(.secondary)

            Text(currentQuestion.text)
                .font(.title2)
                .fontWeight(.bold)
                .multilineTextAlignment(.center)
                .frame(maxWidth: .infinity)
                .padding()
                .background(Color.blue.opacity(0.10))
                .clipShape(RoundedRectangle(cornerRadius: 14))

            ForEach(currentQuestion.options, id: \\.self) { option in
                Button {
                    checkAnswer(option)
                } label: {
                    Text(option)
                        .font(.headline)
                        .frame(maxWidth: .infinity)
                        .padding()
                        .background(Color.gray.opacity(0.15))
                        .foregroundStyle(.primary)
                        .clipShape(RoundedRectangle(cornerRadius: 12))
                }
                .disabled(answerWasChecked)
            }

            if !message.isEmpty {
                Text(message)
                    .font(.subheadline)
```

```
                    .multilineTextAlignment(.center)
                    .foregroundStyle(message.hasPrefix("Correct") ? .green : .red)
            }

            if answerWasChecked {
                Text(currentQuestion.explanation)
                    .font(.footnote)
                    .foregroundStyle(.secondary)
                    .multilineTextAlignment(.center)

                Button(questionNumber == questions.count - 1 ? "See Result" : "Next
Question") {
                    goToNextQuestion()
                }
                .buttonStyle(.borderedProminent)
            }

            Spacer(minLength: 0)
        }
    }
    .padding()
    .navigationTitle("\\(level) Level")
}

private var completionView: some View {
    VStack(spacing: 20) {
        Text("■")
            .font(.system(size: 70))

        Text("Game Complete!")
            .font(.largeTitle)
            .fontWeight(.bold)

        Text("Level: \\(level)")
            .font(.headline)

        Text("Your score: \\(score) / \\(questions.count * 10)")
            .font(.title2)

        Text("Great job! Keep practising your logic skills.")
            .multilineTextAlignment(.center)
            .foregroundStyle(.secondary)

        Button("Play Again") {
            restartGame()
        }
        .buttonStyle(.borderedProminent)

        NavigationLink("Choose Another Level") {
            LevelView()
        }
        .buttonStyle(.bordered)
    }
    .frame(maxWidth: .infinity, maxHeight: .infinity)
}

private func checkAnswer(_ option: String) {
    guard !answerWasChecked else { return }

    if option == currentQuestion.answer {
        score += 10
        message = "Correct! +10 points"
    } else {
        message = "Not quite. Correct answer: \\(currentQuestion.answer)"
    }

    answerWasChecked = true
}

private func goToNextQuestion() {
    if questionNumber + 1 < questions.count {
        questionNumber += 1
        message = ""
        answerWasChecked = false
    } else {
        gameOver = true
    }
}

private func restartGame() {
```

```
        questionNumber = 0
        score = 0
        message = ""
        answerWasChecked = false
        gameOver = false
    }
}

#Preview {
    NavigationStack {
        GameView(level: "Easy")
    }
}
```